

CPU SCHEDULING ALGORITHM

For every scheduling problem, students should remember:

- **Completion Time (CT)** = time at which process finishes
- **Turnaround Time (TAT)** = $CT - AT$
- **Waiting Time (WT)** = $TAT - BT$
- **Response Time (RT)** = First CPU Start Time – AT

Where:

- **AT** = Arrival Time
- **BT** = Burst Time

Problem 1 – FCFS

Given

Process	AT	BT
P1	0	5
P2	1	3
P3	2	4
P4	3	2

Step 1: Apply FCFS

FCFS = First Come, First Served.

Arrange according to arrival time:

P1 → P2 → P3 → P4

Step 2: Gantt Chart

0 5 8 12 14
| P1 | P2 | P3 | P4 |

Step 3: Calculate CT

Process	AT	BT	CT
P1	0	5	5
P2	1	3	8
P3	2	4	12
P4	3	2	14

Step 4: Calculate TAT

$$\text{TAT} = \text{CT} - \text{AT}$$

Process	CT	AT	TAT
P1	5	0	5
P2	8	1	7
P3	12	2	10
P4	14	3	11

Step 5: Calculate WT

$$\text{WT} = \text{TAT} - \text{BT}$$

Process	TAT	BT	WT
P1	5	5	0
P2	7	3	4
P3	10	4	6
P4	11	2	9

Step 6: Response Time

Since FCFS is non-preemptive, the first start time is:

Process	First Start	AT	RT
P1	0	0	0
P2	5	1	4
P3	8	2	6
P4	12	3	9

Final Answer

Average WT

$$(0 + 4 + 6 + 9) / 4 = 4.75$$

Average TAT

$$(5 + 7 + 10 + 11) / 4 = 8.25$$

Problem 2 – FCFS

Given

Process	AT	BT
P1	0	4
P2	2	3
P3	4	2
P4	5	3

Step 1: Order

Arrival order:

P1 → P2 → P3 → P4

Step 2: Gantt Chart

0 4 7 9 12
| P1 | P2 | P3 | P4 |

Step 3–5: Calculations

Process	AT	BT	CT	TAT = CT-AT	WT = TAT-BT	RT
P1	0	4	4	4	0	0
P2	2	3	7	5	2	2
P3	4	2	9	5	3	3
P4	5	3	12	7	4	4

Final Answer

Average WT

$$(0+2+3+4)/4 = 2.25$$

Average TAT

$$(4+5+5+7)/4 = 5.25$$

Problem 3 – SJF Non-Preemptive

Given

All processes arrive at time 0.

Process	BT
P1	6
P2	2
P3	4
P4	3

Step 1: Find shortest burst

Since all arrive at 0, simply arrange according to burst time:

- P2 = 2
- P4 = 3
- P3 = 4
- P1 = 6

Therefore:

P2 → P4 → P3 → P1

Step 2: Gantt Chart

0 2 5 9 15
| P2 | P4 | P3 | P1 |

Step 3: Calculate

Process	AT	BT	CT	TAT	WT	RT
P1	0	6	15	15	9	9
P2	0	2	2	2	0	0
P3	0	4	9	9	5	5
P4	0	3	5	5	2	2

Final Answer

Average WT

$$(9+0+5+2)/4 = 4$$

Average TAT

$$(15+2+9+5)/4 = 7.75$$

Teaching point

In SJF, the process with the **smallest burst time** gets preference.

Problem 4 – SJF Non-Preemptive

Given

Process	AT	BT
P1	0	7
P2	2	4
P3	3	1
P4	5	2

Step 1: At time 0

Only **P1** has arrived.

So CPU executes P1.

$0 \rightarrow 7$

P1

Step 2: At time 7

By time 7, all remaining processes have arrived:

- P2 = 4
- P3 = 1
- P4 = 2

Shortest is **P3**.

Then P4, then P2.

Order:

P1 $\rightarrow$ P3 $\rightarrow$ P4 $\rightarrow$ P2

Step 3: Gantt Chart

0 7 8 10 14
| P1 | P3 | P4 | P2 |

Calculations

Process	AT	BT	CT	TAT	WT	RT
P1	0	7	7	7	0	0
P2	2	4	14	12	8	8
P3	3	1	8	5	4	4
P4	5	2	10	5	3	3

Final Answer

Average WT

$$(0+8+4+3)/4 = 3.75$$

Average TAT

$$(7+12+5+5)/4 = 7.25$$

Important teaching point

At time 0, we **cannot choose P3**, even though P3 has the shortest burst, because **P3 has not arrived yet**.

Ex.Program:

```
#include <stdio.h>

int main() {
    int n, i;
    int bt[10], wt[10], tat[10];
    printf("Enter number of processes: ");
    scanf("%d", &n);

    // Read burst time
    for (i = 0; i < n; i++) {
        printf("Enter Burst Time for P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // Waiting time of first process
    wt[0] = 0;

    // Calculate waiting time
    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }
```

```

// Calculate turnaround time
for (i = 0; i < n; i++) {
    tat[i] = wt[i] + bt[i];
}

// Display results
printf("\nProcess\tBT\tWT\tTAT\n");

for (i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\n",
        i + 1, bt[i], wt[i], tat[i]);
}

return 0;
}

```

Example Input

Enter number of processes: 4

Enter Burst Time for P1: 5

Enter Burst Time for P2: 3

Enter Burst Time for P3: 4

Enter Burst Time for P4: 2

Output

Process	BT	WT	TAT
P1	5	0	5
P2	3	5	8

P3 4 8 12

P4 2 12 14

Types of CPU Scheduling Algorithms

There are two main categories:

1. Non-Preemptive Scheduling

Once a process gets the CPU, it **keeps the CPU until it finishes or enters a waiting state**.

Common algorithms:

- **FCFS (First Come First Serve)**
- **SJF (Shortest Job First)**
- **Non-Preemptive Priority Scheduling**

Example:

If P1 gets the CPU, P2 cannot take the CPU away from P1 until P1 completes.

2. Preemptive Scheduling

The OS can **take the CPU away from a running process** and give it to another process.

Common algorithms:

- **Round Robin (RR)**
- **Shortest Remaining Time First (SRTF)**
- **Preemptive Priority Scheduling**

Example:

P1 is running, but if P2 has higher priority or its time quantum expires, the OS can stop P1 and give the CPU to P2.

2. SJF - Shortest Job First

Definition

SJF selects the process with the shortest CPU burst time among the available processes.

It is normally **non-preemptive**.

Example Problem

Process	Arrival Time	Burst Time
P1	0	7
P2	1	3
P3	2	2
P4	3	4

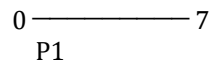
At time 0, only P1 is available, so P1 executes first.

After P1 completes, select the shortest available process.

Step 1: At time 0

Only **P1** has arrived.

So, P1 must execute.



P1 finishes at **time 7**.

Step 2: At time 7

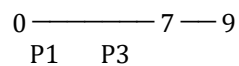
By time 7, all processes have arrived:

- P2 → BT = 3
- P3 → BT = 2
- P4 → BT = 4

SJF chooses the **shortest burst time**.

Therefore:

P3 (BT = 2) is selected.



P3 finishes at **9**.

Step 3: At time 9

Remaining processes:

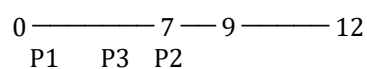
Process BT

P2 3

P4 4

Shortest is **P2**.

So P2 executes from **9 to 12**.



Step 4: At time 12

Only **P4** remains.

Therefore, P4 executes from **12 to 16**.

Final Gantt Chart

0 7 9 12 16
| P1 | P3 | P2 | P4 |

Step 5: Calculate Completion Time (CT)

Completion time is the time at which each process finishes.

Process	AT	BT	CT
P1	0	7	7
P2	1	3	12
P3	2	2	9
P4	3	4	16

Step 6: Calculate Turnaround Time

Formula:

$$\text{TAT} = \text{CT} - \text{AT}$$

P1

$$\text{TAT} = 7 - 0 = 7$$

P2

$$\text{TAT} = 12 - 1 = 11$$

P3

$$\text{TAT} = 9 - 2 = 7$$

P4

$$\text{TAT} = 16 - 3 = 13$$

Process	CT	AT	TAT
P1	7	0	7
P2	12	1	11
P3	9	2	7
P4	16	3	13

Step 7: Calculate Waiting Time

Formula:

$$\text{WT} = \text{TAT} - \text{BT}$$

P1

$$WT = 7 - 7 = 0$$

P2

$$WT = 11 - 3 = 8$$

P3

$$WT = 7 - 2 = 5$$

P4

$$WT = 13 - 4 = 9$$

Process	BT	TAT	WT
P1	7	7	0
P2	3	11	8
P3	2	7	5
P4	4	13	9

Step 8: Average Waiting Time

$$\text{Average WT} = (WT_1 + WT_2 + WT_3 + WT_4) / 4$$

$$= (0 + 8 + 5 + 9) / 4$$

$$= 22 / 4$$

$$= 5.5 \text{ ms}$$

Final Answer

- **Gantt Chart:** P1 → P3 → P2 → P4
- **Average Waiting Time** = 5.5 ms
- **Average Turnaround Time** = $(7 + 11 + 7 + 13) / 4 = 9.5 \text{ ms}$

3. SRTF-Shortest Remaining Time First

Definition

SRTF is the preemptive version of SJF. The process with the shortest remaining burst time gets the CPU.

If a new process arrives with a shorter remaining time, the currently running process can be **preempted**.

Because **1 unit of CPU time has already been used**.

In SRTF, we always track the **remaining burst time**.

For example:

P1

- Burst Time = **8 ms**
- P1 starts at **time 0**
- At **time 1**, P1 has run from **0 to 1**
- Therefore, it has used **1 ms**

So:

Remaining Time = Burst Time – CPU time already used

P1 remaining = 8 – 1 = 7 ms

Think of it like this:

P1 needs **8 seconds** to finish:

Required: 8 seconds

0 → 1

↑ ↑

start 1 second completed

After 1 second:

Completed = 1

Remaining = 8 – 1 = 7

Then P2 arrives at time 1 and preempts P1.

Another example

Suppose:

P1 BT = 10 ms

If P1 runs from **0 to 3**:

CPU used = 3 ms

Remaining = 10 – 3 = 7 ms

If it runs from **0 to 5**:

CPU used = 5 ms

Remaining = 10 – 5 = 5 ms

Important

Remaining Time = Original Burst Time – Total CPU Time Already Executed

So we subtract **1** because P1 has executed for exactly **1 ms**, from time **0 to time 1**.

Example Problem

Consider the following processes:

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	8
P2	1	4
P3	2	2
P4	3	1

We will solve this **step by step**.

Step 1: Time = 0

Only **P1** has arrived.

P1 → Burst Time = 8

So P1 starts executing.

Time: 0

↓
P1

P1 has executed for 1 unit from **0 → 1**.

Remaining time of P1

$$8 - 1 = 7$$

So:

P1 remaining = 7

Step 2: Time = 1

At time 1, **P2 arrives**.

Current processes:

Process	Remaining Time
P1	7
P2	4

SRTF chooses the process with the **shortest remaining time**.

$$P2 = 4 < P1 = 7$$

Therefore, **P1 is preempted**.

P2 starts execution.

0 1
| P1 |
| P2 →

Step 3: Time = 2

At time 2, **P3 arrives**.

P2 has executed for 1 unit.

P2 remaining:

$$4 - 1 = 3$$

Now compare:

Process Remaining Time

P1	7
P2	3
P3	2

Shortest = **P3 (2)**

Therefore, **P2 is preempted**.

P3 starts executing.

0 1 2
| P1 | P2 |
| P3 →

Step 4: Time = 3

At time 3, **P4 arrives**.

P3 has executed for 1 unit.

P3 remaining:

$$2 - 1 = 1$$

Now compare:

Process	Remaining Time
P1	7
P2	3
P3	1
P4	1

There is a **tie** between P3 and P4.

We need a tie-breaking rule.

Let's assume:

If remaining times are equal, continue the currently running process.

Therefore, P3 continues.

P3 executes from **3** → **4** and completes.

```
0  1  2  3  4
| P1 | P2 | P3 | P3 |
```

P3 Completion Time = **4**

Step 5: Time = 4

Remaining processes:

Process Remaining Time

P1	7
P2	3
P4	1

Shortest = **P4**

Therefore, P4 executes.

P4 has BT = 1.

It executes from:

4 → **5**

P4 completes at **5**.

```
0  1  2  3  4  5
| P1 | P2 | P3 | P3 | P4 |
```

Step 6: Time = 5

Remaining:

Process Remaining Time

P1	7
P2	3

Shortest = **P2**

P2 executes from **5** → **8**.

P2 completes at **8**.

0 1 2 4 5 8
| P1 | P2 | P3 | P4 | P2 |

Step 7: Time = 8

Only P1 remains.

P1 has **7 units remaining**.

Therefore P1 executes from:

8 → 15

P1 completes at **15**.

Final Gantt Chart

0 1 2 4 5 8 15
| P1 | P2 | P3 | P4 | P2 | P1 |

So the execution order is:

P1 → P2 → P3 → P4 → P2 → P1

Notice that **P1 and P2 appear more than once**. This happens because SRTF is **preemptive**.

Step 8: Completion Time

Process	AT	BT	CT
P1	0	8	15
P2	1	4	8
P3	2	2	4
P4	3	1	5

Step 9: Turnaround Time

Formula:

TAT = CT – AT

P1

$$15 - 0 = 15$$

P2

$$8 - 1 = 7$$

P3

$$4 - 2 = 2$$

P4

$$5 - 3 = 2$$

Process	CT	AT	TAT
P1	15	0	15
P2	8	1	7
P3	4	2	2
P4	5	3	2

Step 10: Waiting Time

Formula:

$$WT = TAT - BT$$

P1

$$15 - 8 = 7$$

P2

$$7 - 4 = 3$$

P3

$$2 - 2 = 0$$

P4

$$2 - 1 = 1$$

Process	TAT	BT	WT
P1	15	8	7
P2	7	4	3
P3	2	2	0
P4	2	1	1

Step 11: Average Waiting Time

$$\text{Average WT} = (7 + 3 + 0 + 1) / 4$$

$$= 11 / 4$$

$$= 2.75 \text{ ms}$$

Average Turnaround Time

$$\text{Average TAT} = (15 + 7 + 2 + 2) / 4$$

$$= 26 / 4$$

$$= 6.5 \text{ ms}$$

Final Answer

Process	AT	BT	CT	TAT	WT
P1	0	8	15	15	7
P2	1	4	8	7	3
P3	2	2	4	2	0
P4	3	1	5	2	1
Average				6.5	2.75

Whenever a process arrives, compare the remaining time of all available processes. Run the process with the smallest remaining time.

Use this example:

Process	AT	BT
P1	0	8
P2	1	4
P3	2	2
P4	3	1

Step 1 — Draw a timeline first

0 1 2 3 4 5 6 7 8 15
 |---|---|---|---|---|---|---|-----|

Now ask students **only one question at each arrival**:

At time 0

Only P1 is available.

$$P1 = 8$$

So P1 starts.

0 → 1 : P1

P1 has run for 1 unit.

Remaining:

$$\mathbf{P1 = 8 - 1 = 7}$$

At time 1 → P2 arrives

Now compare:

P1 = 7 remaining
P2 = 4

Shortest = **P2**

So **P1 stops** and P2 starts.

1 → 2 : P2

P2 remaining:

$$\mathbf{4 - 1 = 3}$$

Tell students:

"P1 is not completed. It is only paused. We will come back to P1 later."

This is the most important idea in SRTF.

At time 2 → P3 arrives

Compare:

P1 = 7
P2 = 3
P3 = 2

Shortest = **P3**

So P2 is paused.

2 → 3 : P3

P3 remaining:

$$\mathbf{2 - 1 = 1}$$

At time 3 → P4 arrives

Now:

P1 = 7
P2 = 3
P3 = 1
P4 = 1

P3 and P4 are tied.

If we use **FCFS tie-breaking**, P3 continues because P3 was already running/arrived earlier.

So:

3 → 4 : P3

P3 finishes.

At time 4

P3 is completed.

Remaining:

P1 = 7
P2 = 3
P4 = 1

Shortest = **P4**

So:

4 → 5 : P4

P4 finishes.

At time 5

Remaining:

P1 = 7
P2 = 3

Shortest = **P2**

So:

5 → 8 : P2

P2 finishes.

At time 8

Only P1 remains.

P1 = 7

So:

8 → 15 : P1

P1 finishes.

Now construct the Gantt chart

Once students understand the above decisions, **then** show them:

```
0  1  2  3  4  5    8    15
| P1 | P2 | P3 | P3 | P4 | P2 | P1 |
```

You can combine P3's two continuous portions:

```
0  1  2    4  5    8    15
| P1 | P2 | P3 | P4 | P2 | P1 |
```

Tell students to read it like this:

P1 starts → P2 comes and preempts P1 → P3 comes and preempts P2 → P4 comes, but P3 is already shortest → P4 executes → P2 resumes → P1 resumes.

That's the whole SRTF logic.

1. What is Round Robin?

Round Robin is a **preemptive CPU scheduling algorithm**.

The main idea:

- Each process gets a fixed amount of CPU time called **Time Quantum (TQ)**.
- If the process finishes within the quantum → it leaves the ready queue.
- If it does not finish → it is **preempted** and moved to the **back of the ready queue**.
- Then the next process gets the CPU.

Real-life example

Imagine **3 students waiting to use one computer**.

- Student P1 gets **2 minutes**
- Then P2 gets **2 minutes**
- Then P3 gets **2 minutes**
- If P1 needs more time, P1 goes to the **back of the queue**.

This continues until everyone finishes.

2. Simple Problem

Consider:

Process	Arrival Time	Burst Time
P1	0	5

P2	0	4
P3	0	2

Time Quantum = 2 ms

We need to find:

- Gantt Chart
- Completion Time (CT)
- Turnaround Time (TAT)
- Waiting Time (WT)

3. Step 1 — Understand the data

Burst Time

Burst time means:

How much CPU time does the process need in total?

For example:

- P1 needs 5 ms
- P2 needs 4 ms
- P3 needs 2 ms

Time Quantum

Time Quantum = **2 ms**

So each process can run for **maximum 2 ms at a time**.

4. Step 2 — Create the Ready Queue

At time 0, all processes have arrived.

So:

Ready Queue:

P1 → P2 → P3

We start with P1.

5. Step 3 — Execute P1

P1 requires **5 ms**.

Time Quantum = **2 ms**.

So P1 gets only 2 ms.

P1: $5 - 2 = 3$ ms remaining

Time:

$0 \rightarrow 2$

P1 is **not finished**, so put P1 at the back.

Queue becomes:

$P2 \rightarrow P3 \rightarrow P1$

6. Step 4 — Execute P2

P2 requires **4 ms**.

Give P2 = 2 ms.

P2: $4 - 2 = 2$ ms remaining

Time:

$2 \rightarrow 4$

P2 is not finished.

Put P2 at the back.

Queue:

$P3 \rightarrow P1 \rightarrow P2$

7. Step 5 — Execute P3

P3 requires only **2 ms**.

Give P3 = 2 ms.

P3: $2 - 2 = 0$

P3 is finished.

Time:

$4 \rightarrow 6$

Because P3 is finished, **do not put it back into the queue**.

Queue:

$P1 \rightarrow P2$

8. Step 6 — Execute P1 again

P1 has **3 ms remaining**.

Quantum = 2 ms.

Give P1 another 2 ms.

P1: $3 - 2 = 1$ ms remaining

Time:

6 → 8

P1 is not finished.

Put P1 at the back.

Queue:

P2 → P1

9. Step 7 — Execute P2 again

P2 has **2 ms remaining**.

Give P2 2 ms.

P2: $2 - 2 = 0$

P2 finishes.

Time:

8 → 10

Queue: P1

10. Step 8 — Execute P1

P1 has only **1 ms remaining**.

Although quantum = 2 ms, P1 needs only 1 ms.

So it runs for **1 ms**.

P1: $1 - 1 = 0$

P1 finishes.

Time:

10 → 11

All processes are finished.

11. Gantt Chart

The final Gantt chart is:

0 2 4 6 8 10 11
| P1 | P2 | P3 | P1 | P2 | P1 |

Tell students:

“Every process gets 2 ms. If it is not finished, it goes to the back of the queue.”

That's the most important rule.

12. Find Completion Time

Completion Time means:

The time at which the process completely finishes.

From the Gantt chart:

Process	Completion Time
P1	11
P2	10
P3	6

13. Find Turnaround Time

Formula:

TAT = Completion Time – Arrival Time

Here all arrival times are 0.

P1

$$\begin{aligned} \text{TAT} &= 11 - 0 \\ &= 11 \text{ ms} \end{aligned}$$

P2

$$\begin{aligned} \text{TAT} &= 10 - 0 \\ &= 10 \text{ ms} \end{aligned}$$

P3

$$\begin{aligned} \text{TAT} &= 6 - 0 \\ &= 6 \text{ ms} \end{aligned}$$

14. Find Waiting Time

Formula:

WT = Turnaround Time – Burst Time

P1

$$\begin{aligned}\text{WT} &= 11 - 5 \\ &= 6 \text{ ms}\end{aligned}$$

P2

$$\begin{aligned}\text{WT} &= 10 - 4 \\ &= 6 \text{ ms}\end{aligned}$$

P3

$$\begin{aligned}\text{WT} &= 6 - 2 \\ &= 4 \text{ ms}\end{aligned}$$

15. Final Answer

Process	AT	BT	CT	TAT	WT
P1	0	5	11	11	6
P2	0	4	10	10	6
P3	0	2	6	6	4

Average Waiting Time

$$\begin{aligned}\text{Average WT} &= (6 + 6 + 4) / 3 \\ &= 16 / 3 \\ &= 5.33 \text{ ms}\end{aligned}$$

Average Turnaround Time

$$\begin{aligned}\text{Average TAT} &= (11 + 10 + 6) / 3 \\ &= 27 / 3 \\ &= 9 \text{ ms}\end{aligned}$$

16. One important point

Time Quantum does NOT mean every process always runs for exactly 2 ms.

For example, P3 has only 2 ms:

P3 needs 2 ms
Quantum = 2 ms

P3 runs → 2 ms → FINISHED

But P1 has 5 ms:

P1 needs 5 ms

First turn → 2 ms → 3 remaining
Second turn → 2 ms → 1 remaining
Third turn → 1 ms → FINISHED

So the process may get the CPU **multiple times**.

ROUND ROBIN

Time Quantum = 2 ms

P1 → 2 ms → remaining? → YES → back of queue

P2 → 2 ms → remaining? → YES → back of queue

P3 → 2 ms → remaining? → NO → FINISHED

P1 → 2 ms → remaining? → YES → back of queue

P2 → 2 ms → remaining? → NO → FINISHED P1 → 1 ms → remaining? → NO → FINISHED